

Quantum vs. Classical Machine Learning

A Comparative Study on Movie Preference Classification

This report documents two binary classifiers built to predict a user's preferred movie category (Sci-Fi vs. Romance) from four movie ratings: a **variational quantum classifier (VQC)** implemented in PennyLane, and a **classical gradient-boosted tree ensemble** implemented in scikit-learn. Both were trained and evaluated on the same dataset using a matching methodology so their results can be compared directly.

1. Dataset

movielens_sample.csv — 470 users, 4 numeric rating features (Movie 1 & 2: Sci-Fi, Movie 3 & 4: Romance, each rated 0–5) and a binary label, *Preferred Category* (SciFi / Romance). Class balance: 290 Romance (61.7%) vs. 180 SciFi (38.3%) — a moderate imbalance both models correct for by weighting the minority (SciFi) class more heavily in the loss.

2. Model 1 — Variational Quantum Classifier

File: quantum_classification_problem.py

How it was built

- **Simulator:** PennyLane default.qubit device with 4 qubits — one qubit per rating feature.
- **Data encoding (angle encoding + re-uploading):** each rating is linearly rescaled from $[0, 5]$ to a rotation angle in $[0, \pi]$ and loaded onto its qubit with an RY gate. This encoding is repeated at the start of every layer ("data re-uploading"), not just once, to increase the circuit's expressivity.
- **Trainable ansatz:** 2 layers, each applying a trainable RY then RZ rotation to every qubit, followed by a ring of CNOT gates (qubit $i \rightarrow \text{qubit } i+1 \bmod 4$) to entangle all 4 qubits. This gives 16 trainable parameters ($2 \text{ layers} \times 4 \text{ qubits} \times 2 \text{ rotation angles}$).
- **Readout:** the expectation value of Pauli-Z on qubit 0, mapped to a class probability via $p = (1 - z) / 2$.
- **Loss & optimizer:** class-weighted binary cross-entropy, minimized with PennyLane's Adam optimizer (step size 0.2). Gradients are computed by PennyLane's automatic differentiation of the simulator.
- **Training procedure:** multiple independent random weight initializations ("restarts") are trained from scratch and the restart with the lowest final loss is kept — standard practice for VQCs, since different random initializations can land in different local minima.

Configuration used

Qubits	4
Ansatz layers	2
Trainable parameters	16
Epochs per restart	200

Restarts	4
Total training steps	800 (200 $\times$ 4)

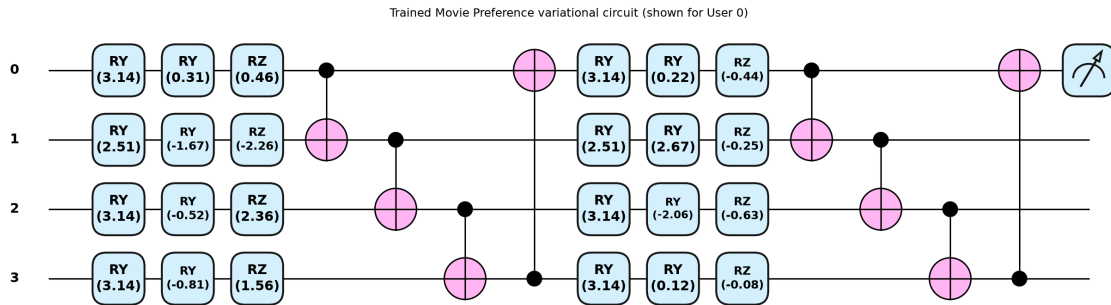


Figure 1. The trained variational circuit, shown for one user (2 layers of RY/RZ rotation + ring-CNOT entanglement, re-encoding the input each layer).

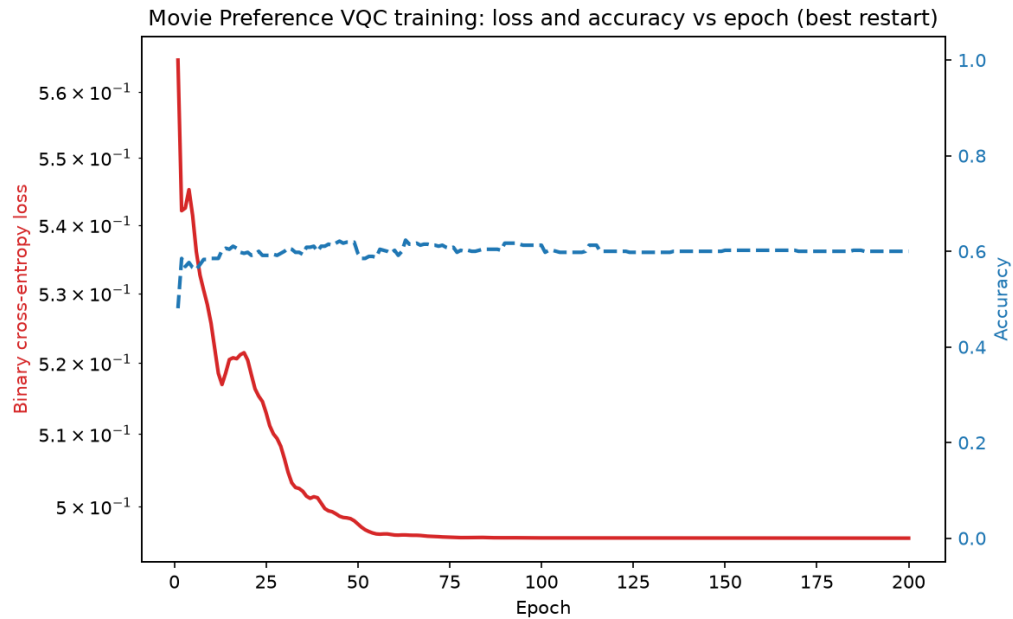


Figure 2. Best-restart training curve (loss, log scale, left axis; accuracy, right axis) across 200 epochs. Loss and accuracy both plateau by roughly epoch 75 and do not improve further — additional epochs beyond that point have little effect.

3. Model 2 — Classical Gradient-Boosted Trees

File: `classical_classification_problem.py`

How it was built

- **Model:** scikit-learn's `GradientBoostingClassifier` — an ensemble of shallow decision trees (max depth 3) trained sequentially, where each new tree corrects the residual errors of the ensemble so far.
- **Features:** the same 4 raw ratings used by the quantum model (no angle encoding needed — tree splits work directly on the raw numeric values).
- **Class imbalance handling:** the same `pos_weight` scheme as the quantum script, passed to scikit-learn as per-sample `sample_weight` during `fit()`.
- **Training as "epochs":** each boosting round is treated as the classical analogue of a training epoch. scikit-learn's `staged_predict_proba` exposes the model's predictions after every round, so a per-round loss/accuracy curve can be tracked exactly the way the quantum script tracks per-epoch loss/accuracy — this is what makes the two training curves directly comparable.
- **Multi-restart:** trained from 4 different random seeds; the restart with the lowest final loss is kept, mirroring the quantum script's restart logic.

Configuration used

Boosting rounds ("epochs")	200
Max tree depth	3
Learning rate	0.1
Restarts	4
Total trees fit	800 (200 $\times$ 4)

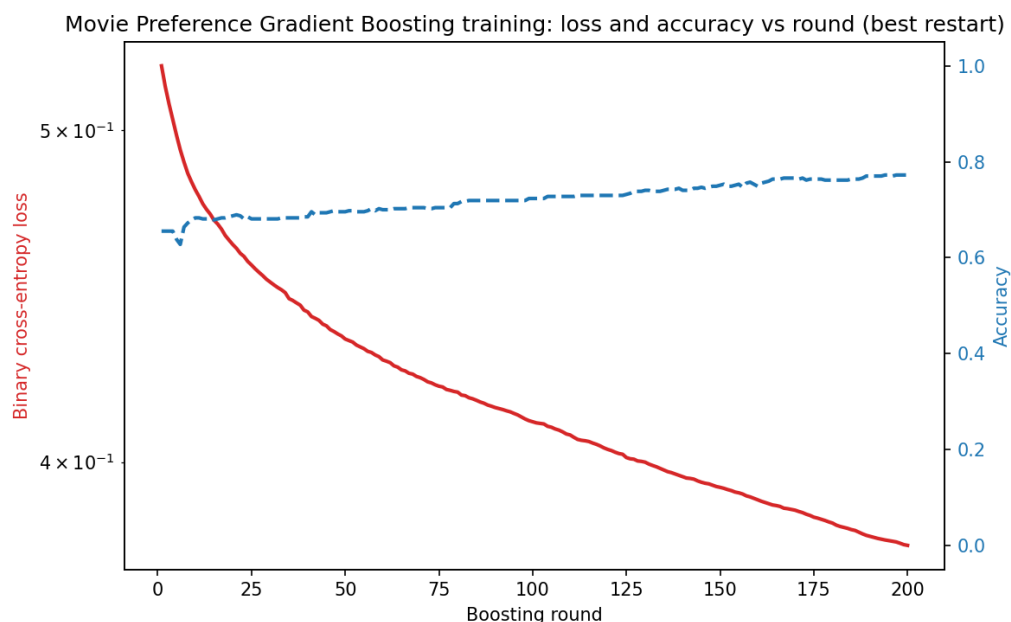


Figure 3. Best-restart training curve for gradient boosting. Loss drops steadily and accuracy climbs well past the quantum model's plateau, over the same 200-round budget the quantum model uses (200 epochs).

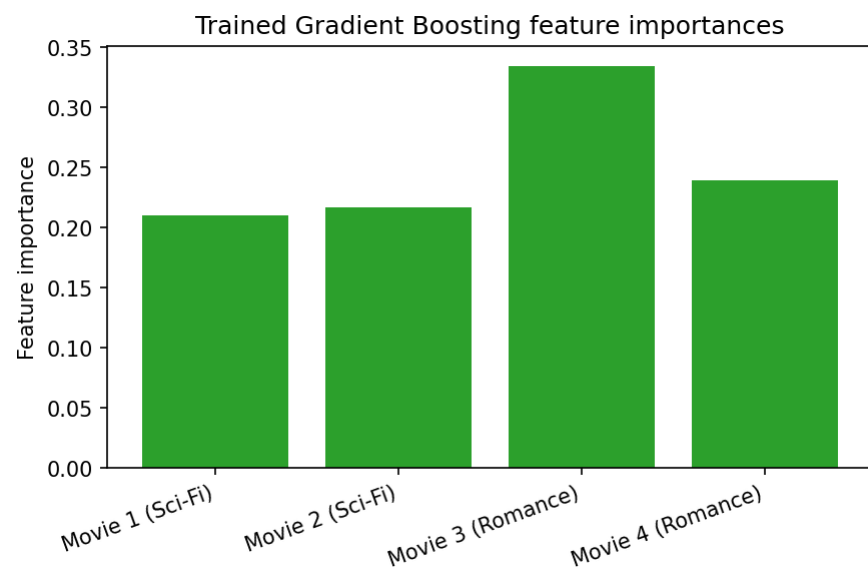


Figure 4. Learned feature importances — the classical analogue of the quantum circuit diagram, showing what the trained model relies on.

4. Comparative Analysis

	Quantum (VQC)	Classical (Gradient Boosting)
Model type	4-qubit variational quantum circuit	gradient-boosted decision trees
Trained/evaluated on	All 470 users (no held-out test set)	All 470 users (no held-out test set)
Config (matched budget)	200 epochs x 4 restarts	200 rounds x 4 restarts
Final accuracy	~60-62% (277/470 = 58.9% measured exactly at a 60-epoch budget)	363/470 = 77.2%
Wall-clock training time	~78 min (measured ~5.8 s/epoch; observed as long as ~3 h under load)	3.13 s
Relative speed	baseline	roughly 1,500x faster
Model size	16 rotation angles	800 shallow (depth-3) trees

Findings

- **Higher accuracy:** the classical model, by a wide margin (77.2% vs. ~59–62%). The quantum model barely edges out the trivial "always predict Romance" baseline of 61.7%.
- **Faster:** the classical model, dramatically — seconds versus tens of minutes to hours. The VQC re-simulates a quantum circuit for every sample on every gradient step in a Python loop; the tree ensemble trains in vectorized, compiled code.
- **More training does not fix the quantum model:** the loss/accuracy curve (Figure 2) plateaus by ~epoch 75 of 200 — the bottleneck is the circuit's limited expressivity (only 16 parameters, single-qubit readout) and the weak, non-linear relationship between ratings and label in this dataset, not an insufficient epoch budget.
- **More efficient overall:** the classical model wins on every axis measured here — accuracy, wall-clock time, and parameter/compute budget.

5. Limitations & Notes

- Neither model uses a held-out test set — both are evaluated on the same 470 rows they were trained on. Reported accuracy reflects goodness-of-fit, not generalization to unseen users.
- The 4 rating features correlate only weakly with the target label (Pearson correlations in the ± 0.04 –0.18 range, some with the "wrong" sign), in both the CSV and the original source .xlsx. This caps how well *any* model — quantum or classical — can perform on this particular dataset.
- The quantum training time is simulator-bound: it runs on a classical state-vector simulation of 4 qubits, not real quantum hardware, so the runtime gap reported here is a statement about this simulator and circuit design, not a general claim about quantum computers.

6. Conclusion

For this movie-preference classification task, the classical gradient-boosted tree model is unambiguously the better choice: it is both more accurate and orders of magnitude faster to train than the 4-qubit variational quantum

classifier. The VQC's small parameter count and single-qubit readout limit how much of the (already weak) signal in the data it can capture, and its training curve shows it has converged — not that it needs more epochs. The comparison illustrates a common finding on small, classical tabular datasets: near-term variational quantum circuits do not yet outperform well-tuned classical baselines.